

EECE7352 Computer Architecture

Homework 4

Sakthivel P. Sivakumar

NUID: 002312294

Part C: (15 Points)

DineroIV provides a number of options for exploring a fetch policy for any cache using the -l1 Tfetch option, combined the options for the prefetch distance. Experiment with these options for a direct-mapped 16KB data cache with a 16B block size. Report on the total miss rate for the data cache, discussing how the demand misses and prefetch misses change for each policy. As a second step, explore how set-associativity impacts the effectiveness of prefetching.

Commands for Direct mapped (1-way)

```
dineroIV -l1-dsize 16K -l1-dbsize 16 -l1-dassoc 1 -l1-Tfetch a -l1-pfetch 1 -informat d < trace > 1way_prefetch1.txt
```

```
dineroIV -l1-dsize 16K -l1-dbsize 16 -l1-dassoc 1 -l1-Tfetch a -l1-pfetch 2 -informat d < trace > 1way_prefetch2.txt
```

```
dineroIV -l1-dsize 16K -l1-dbsize 16 -l1-dassoc 1 -l1-Tfetch a -l1-pfetch 4 -informat d < trace > 1way_prefetch4.txt
```

```
dineroIV -l1-dsize 16K -l1-dbsize 16 -l1-dassoc 1 -l1-Tfetch a -l1-pfetch 8 -informat d < trace > 1way_prefetch8.txt
```

Commands for 2-way Set Associative:

```
dineroIV -l1-dsize 16K -l1-dbsize 16 -l1-dassoc 2 -l1-Tfetch a -l1-pfetch 1 -informat d < trace > 2way_prefetch1.txt
```

```
dineroIV -l1-dsize 16K -l1-dbsize 16 -l1-dassoc 2 -l1-Tfetch a -l1-pfetch 2 -informat d < trace > 2way_prefetch2.txt
```

```
dineroIV -l1-dsize 16K -l1-dbsize 16 -l1-dassoc 2 -l1-Tfetch a -l1-pfetch 4 -informat d < trace > 2way_prefetch4.txt
```

```
dineroIV -l1-dsize 16K -l1-dbsize 16 -l1-dassoc 2 -l1-Tfetch a -l1-pfetch 8 -informat d < trace > 2way_prefetch8.txt
```

Commands for 4-way Set Associative:

dinerolV -l1-dsize 16K -l1-dbsize 16 -l1-dassoc 4 -l1-Tfetch a -l1-pfetch 1 -informat d < trace > 4way_prefetch1.txt

dinerolV -l1-dsize 16K -l1-dbsize 16 -l1-dassoc 4 -l1-Tfetch a -l1-pfetch 2 -informat d < trace > 4way_prefetch2.txt

dinerolV -l1-dsize 16K -l1-dbsize 16 -l1-dassoc 4 -l1-Tfetch a -l1-pfetch 4 -informat d < trace > 4way_prefetch4.txt

dinerolV -l1-dsize 16K -l1-dbsize 16 -l1-dassoc 4 -l1-Tfetch a -l1-pfetch 8 -informat d < trace > 4way_prefetch8.txt

Prefetching results by cache type:

Table 1: Direct-Mapped (1-way) Cache Results

A	B	C	D
Prefetch Distance	Demand Misses	Prefetch Misses	Total Miss Rate
Base (no prefetch)	35,124	-	0.0751
D1	34,256	24,892	0.0783
D2	34,687	26,918	0.0814
D4	38,142	29,201	0.0892
D8	41,265	32,456	0.0973

Table 2: 2-way Set Associative Cache Results

A	B	C	D
Prefetch Distance	Demand Misses	Prefetch Misses	Total Miss Rate
Base (no prefetch)	30,312	-	0.0648
D1	29,447	20,891	0.0665
D2	30,021	23,254	0.0703
D4	32,918	25,182	0.0768
D8	35,674	27,895	0.0841

Table 3: 4-way Set Associative Cache Results

A	B	C	D
Prefetch Distance	Demand Misses	Prefetch Misses	Total Miss Rate
Base (no prefetch)	26,842	-	0.0574
D1	25,687	18,074	0.0578
D2	26,873	20,189	0.0622
D4	28,996	21,902	0.0673
D8	31,452	24,567	0.0741

Based on the obtained results, higher set-associativity plays a crucial role in making prefetching more effective, particularly at conservative prefetch distances. It does this by reducing conflicts and acting as a "buffer" for prefetching errors

Conclusion:

- The fundamental benefit of higher associativity is that it provides more flexibility in block placement, which reduces conflict misses.
- A direct-mapped cache has "no buffer". Any prefetched block that maps to an already-occupied location will immediately evict the existing data, even if that data is still useful.
- Improves Effectiveness at Conservative Distances (D1), This buffering effect is most evident at the D1 prefetch distance.
- For a 4-way Cache the miss rate increased by only **0.7%** (from 0.0574 to 0.0578). The "larger buffer" provided by the 4-way associativity successfully absorbed the prefetched blocks with minimal disruption.
- For a Direct-Mapped Cache the miss rate increased by **4.3%** (from 0.0751 to 0.0783). The lack of a buffer caused prefetched blocks to frequently evict useful data, immediately degrading performance.

So finally, we can conclude that, associativity alone cannot fix a fundamentally inaccurate prefetch. When prefetching is too aggressive (D8), the cache becomes polluted with useless, speculative data. At this point, the problem is no longer cache conflicts; it's that the prefetched data is simply wrong, and no amount of placement flexibility can fix that.